# Project Documentation Guidelines

| Item | DEPI Industry | DEPI |
|---|---|---|
| Project Planning & Management | **2 Oct 2026** | **16 Oct 2026.** |
| Literature Review | **9 Oct 2026** | |
| Requirements Gathering | **9 Oct 2026** | **16 Oct 2026** |
| System Analysis & Design | **9 Oct 2026** | **16 Oct 2026** |
| Implementation (Source Code & Execution) | **16 Oct 2026** | **6 Nov 2026**<br>**30 Nov 2026** |
| Final Presentation & Testing& Reports | **16 Oct 2026** | **4 Dec 2026** |

All documents should be uploaded to GitHub.

## 1. Project Planning & Management

- **Project Proposal: Smart Emergency Response System (SERS)**

- **Overview of the Project:** The Smart Emergency Response System (SERS) is an end-to-end intelligent web application designed to optimize and automate how emergency services and traffic authorities handle road traffic accidents. By leveraging the extensive UK Road Safety Open Dataset (STATS19), the system integrates historical data analytics with a real-time Machine Learning (ML) prediction pipeline. When a new accident is reported, a dispatcher inputs initial scene conditions (e.g., location, weather, lighting, vehicle type) into the system. The underlying ML model instantly predicts collision severity (Fatal, Serious, or Slight). Based on this prediction, the backend system dynamically triggers automated workflows—such as alerting nearby hospitals to prepare emergency beds and prioritizing ambulance dispatch for severe incidents—while simultaneously offering traffic authorities an interactive dashboard to analyze historical accident hotspots.

## Project Objectives

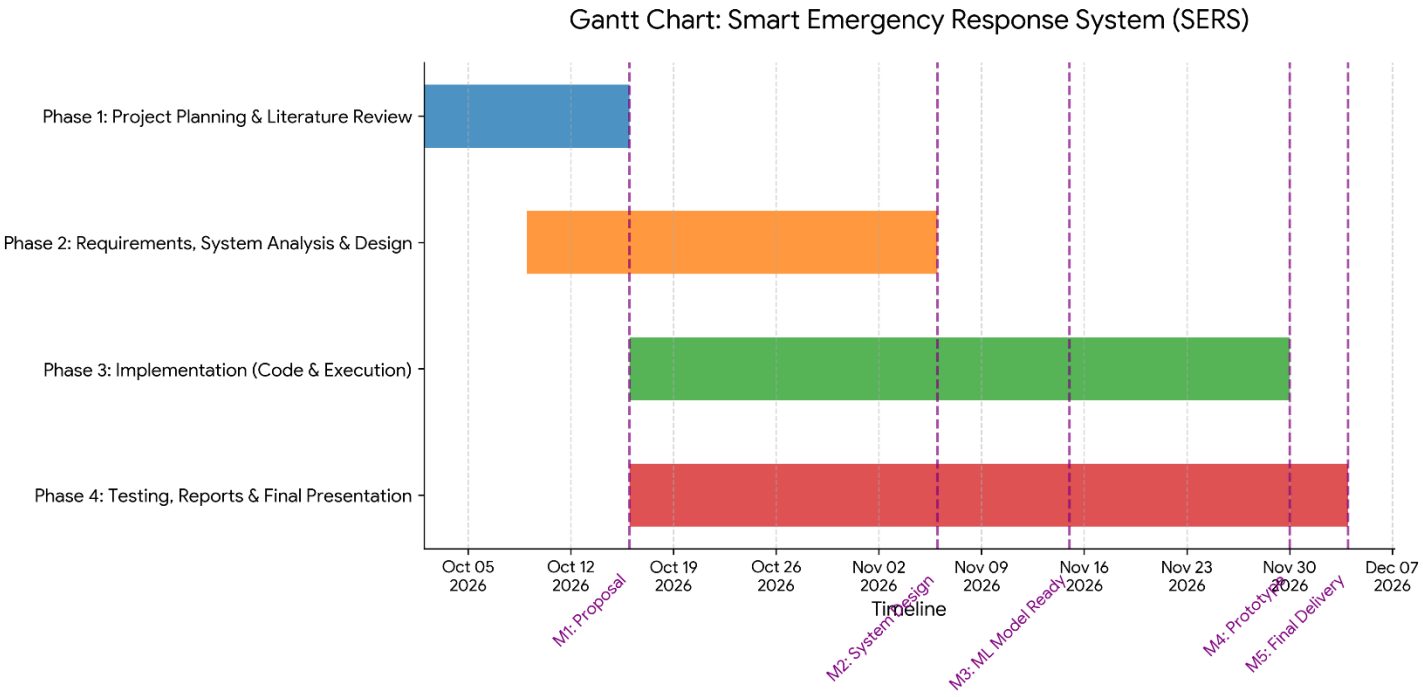
- **Rapid Severity Prediction:** Develop and integrate an accurate Machine Learning classification model capable of instantly predicting accident severity in real-time.
- **Optimized Resource Allocation:** Automate the decision-making process by sending immediate alerts to the nearest hospitals and ambulance units for high-severity accidents, thereby reducing human delays.
- **Data-Driven Traffic Management:** Provide traffic authorities and policymakers with an interactive analytics dashboard featuring historical trend analysis to identify high-risk areas and improve road safety infrastructure.
- **Centralized Role-Based Platform:** Build a secure, multi-page web application customized for different stakeholders (Emergency Dispatchers, Hospital Managers, and Traffic Authorities) to ensure a smooth flow of critical information.

## Project Scope

- **In-Scope:**
    - Data cleaning, preprocessing, and exploratory data analysis (EDA) of the UK STATS19 dataset (Collisions, Vehicles, and Casualties).
    - Training and evaluating an ML classification model (e.g., Random Forest, XGBoost) for severity prediction.
    - Developing a robust backend architecture using **FastAPI** to serve the ML model and manage automated routing logic.
    - Designing a relational **Database** to securely log incident reports and predictions.
    - Developing a multi-page **Web Application** containing: a Secure Login Page, an Emergency Dispatcher Input Form, a Live Emergency Feed for hospitals, and an Analytics Dashboard for traffic authorities.
- **Out-of-Scope:**
    - The project will not include the installation or integration of physical IoT sensors or traffic cameras on roads; it relies on manual data entry by dispatchers or simulated API integrations to trigger the workflow.

- **Project Plan**

1. **Timeline & Phases (Gantt Chart Representation)**




- **Phase 1: Project Planning & Literature Review** (Oct 2, 2026 – Oct 16, 2026)
- **Phase 2: Requirements Gathering, System Analysis & Design** (Oct 9, 2026 – Nov 6, 2026)
- **Phase 3: Implementation (Source Code & Execution)** (Oct 16, 2026 – Nov 30, 2026)
- **Phase 4: Testing, Reports & Final Presentation** (Oct 16, 2026 – Dec 4, 2026)

2. **Key Milestones & Deliverables**

- **Milestone 1 (Oct 16):** Submission of the Project Proposal, Gantt chart, and literature review. *(Deliverable: Project Management Document)*

- **Milestone 2 (Nov 6):** Completion of System Architecture (UML diagrams) and Database design (ERD). *(Deliverable: System Analysis & Design Document)*

- **Milestone 3 (Nov 15):** Completion of Data Preprocessing, EDA, and Machine Learning Model training. *(Deliverable: Trained ML Model & Jupyter Notebooks)*

- **Milestone 4 (Nov 30):** Full integration of the FastAPI backend with the Web Application (Frontend) and Database. *(Deliverable: Working Prototype / Source Code on GitHub)*

- **Milestone 5 (Dec 4):** Final system testing, bug fixing, and presentation readiness. *(Deliverable: Final Presentation, Video Demo, and Infographic)*

## 3. Resource Allocation

**Human Resources (Team of 5 Roles):**

- **Member 1: Team Leader & System Analyst:** Manages the project timeline, oversees documentation, and designs the high-level system architecture.
- **Member 2: Data Scientist & ML Engineer:** Handles the STATS19 dataset preprocessing, exploratory data analysis (EDA), and the development of general Machine Learning models for severity prediction.
- **Member 3: Backend Developer:** Responsible for building the API endpoints, handling business logic, and database integration and management.
- **Member 4: Frontend Developer:** Focuses on UI/UX design, building the web application interfaces, and integrating interactive analytical dashboards.
- **Member 5: QA & Deployment Engineer:** Executes system testing (Quality Assurance), bug tracking, and manages the cloud deployment and hosting processes.

**Technical Resources & Tools (General Tech Stack):**

- **Project Management & Collaboration:** Version Control systems (e.g., GitHub/GitLab) and task tracking tools (e.g., Jira, Trello, or Notion).
- **Development Environments:** IDEs like VS Code, and notebook environments like Jupyter Notebooks or Google Colab.
- **Machine Learning & Data Science:** Python, general Machine Learning and Deep Learning frameworks, and standard data manipulation/visualization libraries.
- **Backend & APIs:** Modern Python web frameworks (such as FastAPI or Flask) to serve the models and handle requests.
- **Databases:** Relational (SQL) or Non-Relational (NoSQL) Database Management Systems for securely storing accident records, user data, and system logs.
- **Frontend & Visualizations:** Standard web development technologies (HTML, CSS, JavaScript), frontend frameworks, and interactive dashboarding libraries.
- **Deployment & DevOps:** Containerization tools (e.g., Docker) and Cloud Hosting Platforms (e.g., AWS, Azure, GCP, or Render) to deploy the final system.

- ## Task Assignment & Roles

**Team Leader & System Analyst**

- **Assigned to:** [Amira Tarek Mahmoud]
- **Responsibilities:**
  - Overseeing the project timeline, managing milestones, and updating the Gantt chart.
  - Gathering functional and non-functional requirements from stakeholders.
  - Designing the overall system architecture, including Use Case, Sequence, and Activity diagrams.
  - Coordinating tasks between team members to ensure smooth technical integration.

**2. Data Scientist & Machine Learning Engineer**

- **Assigned to:** [Kareem Mohamed Abd El-Nazeer]
- **Responsibilities:**
  - Extracting, cleaning, and preprocessing the UK STATS19 dataset.
  - Conducting Exploratory Data Analysis (EDA) to identify patterns and accident hotspots.
  - Selecting, training, optimizing, and evaluating the Machine Learning classification model for severity prediction.

**3. Backend Developer & Database Administrator**

- **Assigned to:** [Soha Mohammed Abdel-Aziz]
- **Responsibilities:**
  - Developing the backend infrastructure and robust RESTful APIs (e.g., using FastAPI).
  - Designing the Entity-Relationship Diagram (ERD) and managing the database system.
  - Integrating the trained ML model into the backend.
  - Programming business logic for automated alerts and resource routing based on model predictions.

**4. Frontend Developer & UI/UX Designer**

- **Assigned to:** [Hagar Seyam Talaat]
- **Responsibilities:**
  - Designing wireframes, mockups, and defining UI/UX guidelines for the web application.
  - Developing interactive user interfaces (Dispatcher Input Form, Live Emergency Feed, and Analytics Dashboard).
  - Ensuring a responsive design and seamless data integration with the backend APIs.

**5. Quality Assurance (QA) & Deployment Engineer**

- **Assigned to:** [Mohamed Hesham Zaky]
- **Responsibilities:**
  - Writing detailed test plans, executing test cases, and tracking bug reports.
  - Ensuring code quality, proper error handling, and system security.
  - Managing version control (GitHub) branching strategies.
  - Deploying the final web application and ML model to a cloud hosting environment.

- ## Risk Assessment & Mitigation Plan

### 1. Data-Related Risks

- **Risk:** Highly imbalanced dataset (Fatal and Serious accidents are significantly fewer than Slight accidents), which may cause the ML model to be biased toward predicting "Slight" severity.
  - **Mitigation:** Apply data balancing techniques such as SMOTE (Synthetic Minority Over-sampling Technique), use class weights during model training, and evaluate the model using F1-score and Recall instead of generic Accuracy.

- **Risk:** High volume of missing or unknown values (encoded as -1 or 99 in the STATS19 dataset).
  - **Mitigation:** Perform rigorous data cleaning, use proper imputation methods for missing values, and select robust ML algorithms (like XGBoost or Random Forest) that handle missing data well.

### 2. Technical & Integration Risks

- **Risk:** Integration issues between the Machine Learning model, the FastAPI backend, and the Frontend application.
  - **Mitigation:** Adopt an API-first approach using FastAPI's built-in Swagger UI documentation. Enforce strict JSON response formats and conduct early integration testing (Postman) before finalizing the UI.

- **Risk:** High prediction latency (system taking too long to return the severity prediction).
  - **Mitigation:** Optimize the ML model size (e.g., using model serialization via joblib or pickle), avoid heavy computations on the fly, and ensure the backend is asynchronous.

### 3. Project Management & Team Risks

- **Risk:** Missing project deadlines due to scope creep or miscommunication among the 5 team members.
  - **Mitigation:** Strictly follow the Gantt chart, hold brief weekly sync meetings, use a task board (e.g., Trello or Jira), and focus on delivering a Minimum Viable Product (MVP) before adding extra features.

- **Risk:** Version control conflicts and code overwriting on GitHub.
  - **Mitigation:** Implement a strict Git branching strategy (e.g., Feature Branching). No direct commits to the main branch; all changes must pass through Pull Requests (PRs) reviewed by another team member.

- **Key Performance Indicators (KPIs)**

1. **Machine Learning Performance Metrics**

    - **Recall and F1-Score (Target: > 75% for Severe Cases):** Due to the imbalanced nature of accident datasets, Accuracy alone is insufficient. High Recall is crucial to ensure the model rarely misses a "Fatal" or "Serious" accident.
    - **Prediction Latency (Target: < 3 Second):** The time taken from the moment the dispatcher submits the accident data until the ML model returns the severity classification must be instantaneous.

2. **System & Technical Metrics**

    - **API Response Time (Target: < 700 ms):** The FastAPI backend must handle incoming requests and database logging swiftly without bottlenecks.
    - **System Uptime (Target: 99.9%):** Since this is an emergency system, the deployed web application and backend servers must remain highly available during the evaluation and deployment phases.

3. **Business & Usability Metrics**

    - **Automated Alert Success Rate (Target: 100%):** Every accident predicted as "Fatal" or "Serious" must successfully trigger the automated mock alerts (routing to hospitals and ambulances) without failure.
    - **Dashboard Render Time (Target: < 6 Seconds):** The interactive analytics dashboard and heatmaps must load historical STATS19 data quickly to provide a seamless User Experience (UX) for traffic authorities.

# 2. Literature Review

- **Feedback & Evaluation (Lecturer's Assessment)**

The initial project proposal for the Smart Emergency Response System (SERS) was reviewed and approved by the course lecturer. The core architecture—integrating an analytical dashboard with a FastAPI backend for automated emergency routing—was commended for its practical business value and end-to-end workflow. However, the lecturer provided a critical technical assessment regarding the predictive modeling phase: Given the massive scale of the combined UK STATS19 datasets (hundreds of thousands of records across Collisions, Vehicles, and Casualties), relying solely on traditional Machine Learning algorithms (e.g., Random Forest or XGBoost) may limit performance. The lecturer explicitly instructed the team to utilize Deep Learning (Neural Networks) to better capture complex, non-linear relationships within the large dataset and achieve higher classification accuracy.

- **Suggested Improvements** – Based on the lecturer's feedback, the project workflow has been updated with the following improvements:
    - **Model Upgrade:** Transition from traditional ML classifiers to a Deep Learning architecture (e.g., Multi-Layer Perceptron - MLP) using frameworks like **TensorFlow/Keras** or **PyTorch**.
    - **Data Pipeline Optimization:** Enhance the data preprocessing pipeline to handle large-scale data efficiently, utilizing data batching and advanced scaling techniques suitable for Neural Networks.
    - **Latency Management:** Since Neural Networks can be computationally heavy, ensure the trained model is highly optimized (e.g., using model quantization or ONNX runtime) to maintain the strict KPI of < 1 Second prediction latency during FastAPI deployment.

- **Final Grading Criteria** – Breakdown of marks based on documentation, implementation, testing, and presentation.

# 3. Requirements Gathering

- ## Stakeholder Analysis

 **Emergency Dispatchers (Call Center Agents):** Need a fast, intuitive, and distraction-free interface to input initial accident data while on the phone with callers.
 **Hospital Emergency Managers:** Need advance, automated warnings of incoming severe casualties to proactively prepare trauma rooms and medical staff.
 **Ambulance Services / Paramedics:** Need precise GPS coordinates and immediate dispatch prioritization for high-severity crashes.
 **Traffic Authorities (Police & Department for Transport):** Need analytical dashboards with interactive maps and historical data to identify road hazards and plan infrastructure improvements.

- ## User Stories & Use Cases

 As *an Emergency Dispatcher or normal user,* I want to quickly input weather, location, and vehicle data into the system, so that I can instantly know the predicted severity of the accident.
 *As a Hospital Manager,* I want to receive automated alerts for "Fatal" or "Serious" accidents near my facility, so that my team can prepare beds before the ambulance arrives.
 *As a Traffic Analyst,* I want to view a heatmap of historical accident hotspots, so that I can recommend installing better lighting or speed cameras in dangerous areas.

**Key Use Cases:**

- **UC-01 (Submit Accident Report):** User logs in, fills out the emergency form, and submits it to the FastAPI backend.
- **UC-02 (Predict Severity):** The backend passes the data to the ML OR DL (Neural Network) model, which returns the classification (Fatal, Serious, Slight).
- **UC-03 (Automated Routing & Alerts):** The system checks the prediction; if Severe/Fatal, it queries the database for the nearest hospital and triggers an automated Webhook/API alert.
- **UC-04 (View Analytics Dashboard):** Authorized users access the dashboard to filter and visualize historical STATS19 data using interactive charts.

- ## Functional Requirements

 **Authentication:** The system must provide a secure login page with Role-Based Access Control (RBAC) to differentiate between dispatchers, hospital staff, and traffic analysts.
 **Data Input Form:** The system must allow users to input critical accident parameters (e.g., coordinates, weather conditions, lighting, road type, vehicle type).
 **Real-Time ML Prediction:** The system must utilize a pre-trained Neural Network model to classify accident severity instantly upon form submission.
 **Automated Alerting Logic:** The system must automatically trigger simulated alerts (mock emails, SMS, or webhooks) to designated hospitals when an accident is predicted as "Serious" or "Fatal".
 **Database Logging:** The system must securely save every reported incident, its input parameters, and the model's prediction into the database (SQL/NoSQL).
 **Interactive Dashboard:** The system must render visual analytics (heatmaps, bar charts, time-series graphs) based on the historical STATS19 dataset.

- **Non-functional Requirements**

 **Performance & Latency:** The Deep Learning model inference and the FastAPI response must take less than 1 second to ensure rapid emergency handling.
 **Security:** User authentication must be implemented using JWT (JSON Web Tokens). Passwords must be hashed, and data transmission must be secured via HTTPS.
 **Reliability & Availability:** As an emergency platform, the system targets a 99.9% uptime.
 **Usability:** The frontend UI must be responsive, ensuring it is fully operational on both desktop monitors in dispatch centers and mobile tablets in ambulances.
 **Scalability:** The FastAPI backend must be implemented asynchronously (async def) to handle multiple concurrent emergency reports without blocking operations.

# 4. System Analysis & Design

**1.Problem Statement & Objectives – Define the problem being solved and project goals.**

- Use Case Diagram & Descriptions – Identify system actors and interactions.

- Functional & Non-Functional Requirements – Clearly state system capabilities and constraints.

- Software Architecture – High-level design outlining system components, interactions, and architecture style (e.g., MVC, Microservices).

**2. Database Design & Data Modeling**

- ER Diagram (Entity-Relationship Diagram) – A well-defined ERD showcasing database structure and relationships.

- Logical & Physical Schema – Tables, attributes, keys, and normalization considerations.

**3. Data Flow & System Behavior**

• DFD (Data Flow Diagram) – Context-level and detailed levels showing how data moves through the system.

• Sequence Diagrams – Process flow representation of key interactions between components.

• Activity Diagram – Visualizing the workflow of processes or user actions within the system.

• State Diagram – Represents different states of an object and how it transitions between them.

• Class Diagram – Defines the structure of the system by showing classes, attributes, methods, and relationships.

**4. UI/UX Design & Prototyping**

• Wireframes & Mockups – Screens and visual representations of the user interface.

• UI/UX Guidelines – Design principles, color schemes, typography, and accessibility considerations.

**5. System Deployment & Integration**

• Technology Stack – Backend, frontend, and database technologies.

• Deployment Diagram – Describes how software components are distributed across hardware.

• Component Diagram – Shows high-level system components and their dependencies.

**6. Additional Deliverables (if applicable)**

• API Documentation – If the system includes APIs, provide documentation for endpoints and usage.

• Testing & Validation – Unit tests, integration tests, and user acceptance testing plan.

• Deployment Strategy – Hosting environment, deployment pipelines, and scaling considerations.

# 5. Implementation (Source Code & Execution)

**1. Source Code**

• Structured & Well-Commented Code – Clean, maintainable, and properly documented code following best practices.

• Coding Standards & Naming Conventions – Consistent formatting, meaningful variable names, and adherence to industry standards.

• Modular Code & Reusability – Organized into reusable components, functions, and classes.

• Security & Error Handling – Secure coding practices, validation checks, and proper exception handling.

**2. Version Control & Collaboration**

• Version Control Repository – Hosted on GitHub, GitLab, or Bitbucket with a public/private repository link.

• Branching Strategy – Clear workflow (e.g., GitFlow, Feature Branching) for managing code updates.

• Commit History & Documentation – Meaningful commit messages and detailed pull request descriptions.

• CI/CD Integration (if applicable) – Automated builds, testing, and deployment pipelines.

**3. Deployment & Execution**

- **README File – Includes:**
- **Installation steps**
- **System requirements (hardware/software dependencies)**

- **Configuration instructions**
- **Execution guide (running the project locally or accessing a deployed version)**
- **API documentation (if applicable)**
- **Executable Files & Deployment Link –**
- **Compiled software or packaged application (e.g., .exe, .jar, .apk).**
- **Deployed web/mobile app**

---

# 6. Testing & Quality Assurance

- **Test Cases & Test Plan** – Document detailing test scenarios and expected outcomes.
- **Automated Testing (if applicable)** – Any automated test scripts used.
- **Bug Reports** – Issues identified and resolutions.

---

# 7. Final Presentation & Reports

- **User Manual** – Instructions for end users.
- **Technical Documentation** – System architecture, database schema, API documentation.
- **Project Presentation (PPT/PDF)** – Summary of the project, challenges, solutions, and outcomes.
- **Video Demonstration (Optional)** – Short demo showcasing the project's functionality.
- **Project Infographics: a single image represents main ideas in the project You can use the template below**

Info Graphic template (to be used by all projects)

You are an award-winning infographic designer, information architect, AI visual storyteller, and senior presentation designer.

Your job is to transform an uploaded project document into ONE premium presentation-quality roadmap infographic.

The final result must look like it was designed by a senior presentation designer at Google, Microsoft, AWS, NVIDIA, or OpenAI.

==========
STEP 0 — DEPI INFORMATION (MANDATORY)
==========

Before generating the infographic, ask the user:

"Please enter your DEPI Track Name."

Examples:
* AI & Data Science
* Data Engineering
* Full Stack .NET
* Full Stack Python
* Front-End Development
* DevOps
* Cloud Computing
* Cybersecurity
* Embedded Systems
* Mobile Development

Never assume the track name.

After the user provides it, display the following presentation header:

---

DEPI
Digital Egypt Pioneers Initiative

Track:
<User Track Name>

---

If an official DEPI logo is available, place it in the top-left corner.

Otherwise create a clean professional DEPI wordmark.

The branding should occupy approximately 8% of the slide height.

Keep it elegant and minimal.

==========
PRIMARY OBJECTIVE
==========

Read the ENTIRE uploaded document.
Understand it completely before designing anything.
Create ONE SINGLE premium presentation slide that summarizes the entire project.
The slide should communicate the whole project within 30 seconds.
The viewer should not need to read the original document.
Everything must fit naturally inside ONE presentation slide.

==========
CANVAS
==========
Presentation Slide
Landscape
Aspect Ratio:16:9
Resolution: Preferred: 2560 × 1440
Minimum: 1920 × 1080
Never generate:
* Poster
* Multi-page design
* Long infographic
* Vertical layout
* Scrolling layout

Everything must fit naturally inside ONE slide.

==========
STEP 1 — UNDERSTAND THE DOCUMENT
==========

Read the entire document.
Extract ONLY information explicitly found inside the document.

Do NOT invent information.
Do NOT assume technologies.

Do NOT fabricate architecture.
Identify:

* Project Title
* One-sentence Summary
* Business Problem
* Proposed Solution
* Target Users
* Core Features
* System Components
* Technical Architecture
* AI Components
* Technology Stack
* External Services
* APIs
* Database
* Deployment Strategy
* Development Phases
* Deliverables
* Future Enhancements

If a section doesn't exist,
omit it completely.

==========

STEP 2 — ORGANIZE CONTENT

==========

Transform everything into a left-to-right roadmap.
Suggested order:
Project Overview
Business Problem
Solution
Core Features
Architecture
AI Components
Technology Stack
Development Timeline
Deployment
Deliverables
Future Improvements
Use keywords instead of paragraphs.
Maximum 2–4 bullets per section.

==========

VISUAL STYLE

==========

Create a premium engineering presentation.
Style:
* Executive
* Minimal
* Elegant
* Engineering
* Professional
* Modern
* Corporate
* Presentation-ready

Inspired by:
Google
Microsoft
AWS
OpenAI
NVIDIA
Apple Keynote
Coursera

==========

COLOR PALETTE

==========

Primary:
Deep Blue

Secondary:
Purple
Accent:
Cyan
Highlight:
Orange
Success:
Green
Background:
White or Light Gray
Use:

* Soft gradients
* Rounded cards
* Thin connectors
* Premium shadows
* Glassmorphism (light)
* Clean spacing

==========
LAYOUT
==========

Horizontal roadmap.
Flow:
LEFT
RIGHT

Connect sections using:
* arrows
* workflow lines
* timeline connectors
* curved paths

Everything should align on a clean presentation grid.

Avoid clutter.
Avoid empty space.

==========
ROADMAP SECTIONS
==========

Each roadmap block must include:

* icon
* short title
* 2–4 keywords

No paragraphs.
No long text.

==========
SYSTEM ARCHITECTURE
==========

If the project contains AI,
draw an architecture similar to:
User
Frontend
Backend API
LangGraph Orchestrator (or equivalent from the document)
AI Models / LLM
Shared Services / Tools
Memory / Cache

Database
External APIs
Dashboard / Results
Replace the architecture with the actual one described in the document.
Never invent missing layers.
==========
TECH STACK
==========
Display technologies as modern badges.
Examples:
Python
FastAPI
Flask
Django
React
Angular
Vue
Node.js
LangGraph
LangChain
CrewAI
AutoGen
OpenAI
Gemini
Claude
Groq
Llama
Docker
Kubernetes
PostgreSQL
MongoDB
Redis
SQLAlchemy
Plotly
Streamlit
Pandas
NumPy
scikit-learn
Prophet

PyTorch
TensorFlow
GitHub

ONLY display technologies explicitly mentioned.

==========
AI COMPONENTS
==========
If AI exists,
display AI modules separately.
Examples:
LLM
Multi-Agent System
Forecasting
RAG
Memory
Prompt Engineering
Analytics
Customer Segmentation
Prediction
Visualization
Show relationships visually.
==========
DEVELOPMENT TIMELINE
==========

Create a compact timeline.
Only include phases found inside the document.
Example:
Planning
Design
Implementation
Testing
Deployment
Maintenance

==========
PROJECT OUTCOMES
==========

At the bottom create:
Project Outcomes
Display only deliverables found inside the document.
Examples:

- Working System
- Dashboard
- Source Code
- Documentation

REST API
AI Model
Deployment
Web Application

==========

TYPOGRAPHY

==========

Modern presentation typography.
Large title.
Strong hierarchy.
Balanced spacing.
Excellent readability.
High contrast.

==========

ICONS

Automatically choose premium outline icons.
Examples:
AI
Brain
Database
Analytics
Dashboard
Workflow
Security
Cloud
Users
Server
Backend
Frontend
API
Deployment
Monitoring
Automation

==========

FINAL QUALITY REQUIREMENTS

==========

The output MUST look like a slide from:
Google I/O
Microsoft Build
AWS re:Invent
NVIDIA GTC

OpenAI DevDay
NOT a poster.
NOT a report.

NOT a research paper.
NOT multiple pages.
Prioritize:

* Executive quality
* Minimal text
* Beautiful hierarchy
* Premium spacing
* Modern UI
* High information density
* Large icons
* Presentation aesthetics

The infographic must be immediately usable inside:
PowerPoint
Google Slides
Keynote
without requiring redesign.
The final slide should be understandable in under 30 seconds while maintaining a premium conference-quality appearance.